

JAVA INTERVIEW PREPARATION

CHAPTER 23

LOCKS, ATOMICS, AND COORDINATION

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Locks, Atomics, and Coordination Utilities: Selecting the Right Concurrency Primitive

Senior / Lead Java Developer Reading Chapter

Java provides explicit locks, atomic variables, semaphores, latches, barriers, and other coordination tools because concurrency requirements differ. The goal is not to use the most advanced primitive. It is to choose the smallest mechanism that clearly protects the invariant, handles cancellation, and behaves predictably under contention and failure.

ReentrantLock Versus synchronized

ReentrantLock and synchronized both provide mutual exclusion and memory visibility. ReentrantLock adds operations that built-in monitors do not expose directly: interruptible acquisition, timed acquisition, optional fairness, multiple Condition objects, and explicit lock inspection.

```
private final ReentrantLock lock = new ReentrantLock();

void update() {
    lock.lock();
    try {
        updateState();
    } finally {
        lock.unlock();
    }
}
```

The finally block is mandatory. An exception that skips unlock can permanently block other callers. Use synchronized when its structured syntax expresses the requirement; choose ReentrantLock for a capability the design actually needs.

Timed and Interruptible Acquisition

Explicit locks can avoid waiting forever.

```
if (!lock.tryLock(200, TimeUnit.MILLISECONDS)) {
    throw new BusinessException("Resource is currently unavailable");
}
try {
    updateState();
} finally {
    lock.unlock();
}
```

`lockInterruptibly` allows cancellation while waiting. Timeout and interruption policies must propagate through the call chain; returning a misleading success after failing to acquire the lock corrupts business meaning.

Fair locks broadly favor the longest-waiting thread, but fairness reduces throughput and still does not promise application-level scheduling fairness. Use it only when starvation evidence and latency requirements justify the cost.

Conditions: More Than One Wait Set

A Condition is associated with a Lock and supports await and signal operations. Multiple conditions can represent different predicates under one invariant.

```
private final ReentrantLock lock = new ReentrantLock();
private final Condition notEmpty = lock.newCondition();
private final Condition notFull = lock.newCondition();
```

Like monitor wait, await releases the lock and reacquires it before returning. Always await in a loop.

```
while (queue.isEmpty()) {
    notEmpty.await();
}
```

In application code, a tested BlockingQueue is normally safer than implementing a bounded queue manually. Conditions matter when a custom state machine truly requires them.

ReadWriteLock and Workload Reality

ReentrantReadWriteLock permits concurrent readers while writers are exclusive.

```
read lock: R1 + R2 + R3 may coexist
write lock: W excludes readers and other writers
```

It can help when reads are frequent and sufficiently long, writes are rare, and contention is measurable. For small critical sections, its coordination overhead may exceed any benefit. Readers can also delay writers, depending on policy and traffic.

```
readLock.lock();
try {
    return index.get(key);
} finally {
    readLock.unlock();
}
```

Lock upgrading from read to write is dangerous and can deadlock. Release the read lock and obtain the write lock, then recheck the condition because state may have changed.

StampedLock and Optimistic Reads

StampedLock supports write locks, read locks, and optimistic reads. An optimistic read does not exclude a writer, so every observed state must be validated before use.

```
long stamp = lock.tryOptimisticRead();
double currentX = x;
double currentY = y;
if (!lock.validate(stamp)) {
    stamp = lock.readLock();
    try {
        currentX = x;
        currentY = y;
    } finally {
        lock.unlockRead(stamp);
    }
}
```

StampedLock is not reentrant. Stamps must be paired with the correct unlock operation, and optimistic reads are unsuitable when a temporarily inconsistent read can itself cause failure before validation. It is a specialized optimization, not a default replacement for synchronized.

Compare-and-Set and Atomic Variables

Atomic classes implement operations such as compare-and-set, conceptually:

```
if current value equals expected:
    replace it with update and report success
else:
    report failure
```

```
AtomicInteger remaining = new AtomicInteger(10);

boolean reserveOne() {
    while (true) {
        int current = remaining.get();
        if (current == 0) return false;
        if (remaining.compareAndSet(current, current - 1)) return true;
    }
}
```

The loop retries when another thread wins the race. Lock-free does not mean wait-free: one thread may repeatedly fail, and heavy contention can waste CPU. Use higher-level atomic methods such as `updateAndGet` when they express the operation safely.

Atomicity Has a Boundary

An `AtomicReference` makes reference replacement atomic, not arbitrary mutation of the referenced object.

```
AtomicReference<List<String>> names =
    new AtomicReference<>(List.of());
```

Mutating a shared `ArrayList` stored in the reference remains unsafe. Instead, construct a new immutable value and compare-and-set the entire snapshot.

`AtomicInteger` fields also do not make a multi-field invariant atomic. If debit and credit must move together, independent atomic updates can expose an impossible intermediate state. Use one immutable aggregate in an `AtomicReference`, a lock around the aggregate, or an authoritative transaction.

The ABA Problem

Compare-and-set can observe that a value equals A without knowing it changed from A to B and back to A.

```
Thread 1 reads A
Thread 2 changes A -> B -> A
Thread 1 CAS expects A and succeeds
```

Whether ABA matters depends on what the value represents. `AtomicStampedReference` or versioned immutable state can include a generation. Many application counters do not suffer a meaningful ABA problem, while lock-free linked structures may.

LongAdder and Contended Statistics

`AtomicLong` updates one logical location, which can become a contention hotspot. `LongAdder` spreads updates across cells and combines them for sum.

```
LongAdder requests = new LongAdder();
requests.increment();
long observed = requests.sum();
```

The observed sum need not be an atomic snapshot with concurrent increments. `LongAdder` is excellent for metrics and frequency counts, but wrong for a strict account balance, sequence, or capacity decision.

Semaphore: Protecting Scarce Capacity

A Semaphore controls concurrent permits rather than assigning work to a small thread pool.

```
private final Semaphore permits = new Semaphore(50);

Result callDependency() throws InterruptedException {
    permits.acquire();
    try {
        return client.call();
    } finally {
        permits.release();
    }
}
```

The finally block prevents permit leaks. A timeout may be better than indefinite acquire. Semaphores are especially useful with virtual threads: threads remain cheap while downstream concurrency stays bounded.

A permit limits simultaneous access, not rate per second. Rate limiting needs a time-based algorithm such as token bucket, while a connection pool also owns actual reusable connections.

CountDownLatch, CyclicBarrier, and Phaser

CountDownLatch allows threads to wait until a fixed number of events complete. It is one-shot.

```
CountDownLatch ready = new CountDownLatch(workerCount);
// each worker calls countDown; coordinator calls await
```

CyclicBarrier coordinates a fixed group reaching repeated phases and can run a barrier action. Phaser supports dynamic registration and multiple phases, but its flexibility adds conceptual complexity.

```
CountDownLatch: events count down once -> gate opens permanently
CyclicBarrier: fixed parties meet -> barrier resets
Phaser: dynamic parties advance through numbered phases
```

These utilities coordinate milestones; they do not automatically propagate task failure. A worker that exits before countDown can leave a coordinator waiting forever unless finally blocks, timeouts, and failure channels are designed.

Exchanger and Blocking Queues

Exchanger lets two threads swap values at a rendezvous and fits narrow pipeline designs. BlockingQueue is more common: it combines safe publication, waiting, and often bounded capacity for producers and consumers.

```
BlockingQueue<Job> queue = new ArrayBlockingQueue<>(1_000);
queue.put(job);           // can block when full
Job next = queue.take();  // blocks when empty
```

Choose offer with timeout or rejection when blocking an incoming request is unacceptable. Queue capacity is an operational policy, not merely a constructor detail.

Interruption Is Cooperative Cancellation

Interrupting a thread sets its interrupt status and causes certain blocking methods to throw InterruptedException. It does not forcibly terminate arbitrary code.

```
try {
    permits.acquire();
} catch (InterruptedException exception) {
    Thread.currentThread().interrupt();
    throw new OperationCancelledException(exception);
}
```

If a method can declare `InterruptedException`, propagating it is often best. When translating it, restore status unless the method intentionally consumes the cancellation signal. Long CPU loops must check interruption if cancellation is required.

Choosing by Invariant

one compound mutable invariant	synchronized or Lock
independent published flag/reference	volatile
single atomic numeric/reference op	atomic class
high-contention approximate metric	LongAdder
limit simultaneous dependency calls	Semaphore
producer-consumer handoff	BlockingQueue
wait for fixed completion events	CountDownLatch
repeated phase rendezvous	barrier or Phaser

Do not combine primitives casually. A lock plus atomics plus volatile fields around the same state often indicates that the ownership model is unclear.

The Queue Behind Most Explicit Synchronizers

Many Java synchronizers can be understood as two things: a small piece of state and a queue of threads that could not change that state immediately. [ReentrantLock](#), [Semaphore](#), [CountDownLatch](#), and several other utilities are built on [AbstractQueuedSynchronizer](#) (AQS). You rarely subclass AQS in application code, but its model explains what the higher-level utilities are doing.

```

      attempt to acquire
Thread A -----> synchronizer state
                        | available: proceed
Thread B ---- acquisition fails -----> unavailable
      |
      +--> queued --> parked --> signalled -----+
```

Parking is different from continuously spinning. A parked platform thread does not consume a CPU core while it waits, although queue management, context switching, cache invalidation, and wake-up latency still have costs. Atomics often spin and retry for a short operation; locks usually queue and park when an operation cannot proceed. Neither strategy is universally faster. The important variables are contention, critical-section duration, scheduler behavior, and the cost of retrying.

This model also explains why a synchronizer can appear healthy at low load and then show a nonlinear latency increase. Once arrival rate exceeds the protected resource's service rate, the queue grows. Average lock hold time may remain small while tail waiting time becomes unacceptable.

Fairness, Barging, and Throughput

A fair [ReentrantLock](#) generally gives preference to the longest-waiting thread. A nonfair lock permits barging: a newly arriving thread may acquire the lock before a thread already queued. Barging sounds undesirable, but it can improve throughput because a running thread may reuse a hot CPU cache and avoid another scheduling transition.

Fair policy:	queued A -> queued B -> new C	acquisition: A, B, C
Nonfair policy:	queued A -> queued B -> new C	possible: C, A, B

Fairness is therefore a policy with a measurable cost, not a moral default. It may reduce variance or starvation risk in a specific workload, but it cannot guarantee fairness for the entire application: the operating-system scheduler, downstream pools, database locks, and network queues have their own policies. Also note an easily missed API detail: the untimed `tryLock()` may acquire a fair `ReentrantLock` even when other threads are waiting. If ordering is part of correctness rather than merely performance, an ordinary lock fairness flag is usually not a sufficient protocol.

Unlock Discipline and Failure Safety

An explicit lock is not automatically released when a block ends. The safe shape is deliberately repetitive:

```
lock.lock();                // acquire before entering try
try {
    mutateProtectedState();
} finally {
    lock.unlock();
}
```

The acquisition occurs before `try` because `unlock()` must run only if this path acquired the lock. With `tryLock`, record acquisition explicitly:

```
boolean acquired = lock.tryLock(50, TimeUnit.MILLISECONDS);
if (!acquired) {
    throw new BusinessException("inventory partition is busy");
}
try {
    reserveInventory();
} finally {
    lock.unlock();
}
```

Condition waiting adds another state transition. `await()` atomically releases the associated lock and parks; before it returns it reacquires that lock. The predicate must still be checked in a loop because wake-ups can be spurious and another thread can consume the condition before the awakened thread runs. A signal means "the state may now permit progress," not "your operation is guaranteed to succeed."

Read-Write Locks: Upgrades, Downgrades, and Starvation

A read-write lock allows multiple readers together but excludes writers from both readers and other writers. That only helps if reads are frequent, read sections are long enough to justify the extra machinery, and writes are uncommon enough not to dominate. For tiny lookups, a concurrent collection or immutable snapshot is often simpler and faster.

Upgrading while retaining a read lock is dangerous. Two readers can both decide that a write is necessary, then both wait forever for all readers to leave. The safe general pattern is to release the read lock, acquire the write lock, and recheck the predicate because the state may have changed in between.

Downgrading is possible when an implementation supports it: acquire the write lock, make the change, acquire the read lock while still holding the write lock, then release the write lock. The read lock preserves a stable view. This is subtle code and needs a clear invariant and tests. In production, observe writer wait time separately; a high stream of readers can create unacceptable writer latency depending on policy and workload.

StampedLock Validation Is the Safety Boundary

An optimistic read does not actually lock out a writer. It takes a stamp, copies fields, and then validates that no conflicting write occurred during the copy. The copied values must not escape before validation.

```
double distanceFromOrigin() {
    long stamp = lock.tryOptimisticRead();
    double currentX = x;
    double currentY = y;

    if (!lock.validate(stamp)) {
        stamp = lock.readLock();
        try {
            currentX = x;
            currentY = y;
        } finally {
            lock.unlockRead(stamp);
        }
    }
    return Math.hypot(currentX, currentY);
}
```

The first copies are speculative. Only successful validation makes the snapshot usable; otherwise the code falls back to a real read lock. A **StampedLock** is non-reentrant, its stamps are capabilities required for unlock or conversion, and it does not use thread ownership in the same way as **ReentrantLock**. Calling a method that tries to reacquire the same mode can therefore deadlock. Use it only when profiling shows that optimistic reads materially help and the team can maintain the stricter protocol.

CAS Loops and the Linearization Point

Compare-and-set is most understandable when an immutable state object holds the entire invariant. The operation reads one snapshot, derives a replacement, and publishes it only if the snapshot is still current.

```
record AccountState(long version, long balanceInCents) {}

final AtomicReference<AccountState> state =
    new AtomicReference<>(new AccountState(0, 0));

void deposit(long amount) {
    if (amount <= 0) throw new IllegalArgumentException();

    while (true) {
        AccountState before = state.get();
        AccountState after = new AccountState(
            before.version() + 1,
            Math.addExact(before.balanceInCents(), amount));
        if (state.compareAndSet(before, after)) {
            return;
        }
    }
}
```

The successful CAS is the operation's linearization point: the instant at which the concurrent history can treat the deposit as having occurred. Failed attempts have not committed. Consequently the retry body must not send email, charge a card, publish a message, or perform any other non-repeatable side effect. Compute a candidate state inside the loop; perform carefully designed follow-up work only after success.

CAS-based code is often lock-free, meaning system-wide progress is guaranteed despite a delayed participant. That does not make it wait-free: one unlucky thread can repeatedly lose and retry. Heavy contention can turn retries and cache-line traffic into high CPU. A lock may be a better engineering choice for a larger state transition or when fairness and bounded waiting matter.

ABA and Versioned State

CAS normally asks whether the current value is equal to the expected value. It cannot tell whether the value changed and later changed back.

```
Thread A reads node A
Thread B changes A -> B -> A
Thread A sees A again and CAS succeeds
```

If identity alone captures the full meaning, this may be harmless. In a lock-free stack or reusable resource pool, the intervening history can invalidate assumptions. Pairing the value with a monotonically changing version, for example through [AtomicStampedReference](#) or an immutable state containing a version, lets CAS compare both. The senior-level question is not merely "can ABA happen?" but "does an A-to-B-to-A history change the validity of my derived operation?"

Permit Accounting Is an Ownership Protocol

A semaphore is a counter, not an owner-aware lock. Any thread can call `release()`, even if it did not acquire. That flexibility makes permit accounting an application responsibility.

```
boolean acquired = downstreamSlots.tryAcquire(100, TimeUnit.MILLISECONDS);
if (!acquired) throw new BusyException("dependency capacity exhausted");
try {
    callDependency();
} finally {
    downstreamSlots.release();
}
```

A missing release leaks capacity until the process may stop making progress. An extra release inflates capacity and silently defeats the protection. Metrics should therefore include configured permits, available permits, wait duration, timeout count, and in-flight work. A semaphore limits concurrency, not requests per second; rate limiting is a different time-based policy.

Coordination Utilities Encode Different Lifecycles

Choosing a utility starts with the lifecycle of the event, not with a familiar class name.

<code>CountDownLatch</code>	one gate, opened after N events; cannot reset
<code>CyclicBarrier</code>	fixed parties meet repeatedly at phase boundaries
<code>Phaser</code>	repeated phases with dynamic registration/deregistration
<code>Semaphore</code>	N permits bound simultaneous access
<code>BlockingQueue</code>	ownership and backpressure between producers/consumers
<code>Exchanger</code>	two parties swap one value at a rendezvous

A `CountDownLatch` fits "start after configuration and three caches are ready" or "wait for ten fixed tasks." A `Phaser` is better when participants join and leave across multiple stages. A bounded `BlockingQueue` expresses both safe publication and capacity. These lifecycle differences matter during failure: ask what happens if a participant times out, throws, is cancelled, or never arrives.

A Strong Interview Explanation

A strong answer begins with the protected invariant and progress requirement. Use `synchronized` for straightforward mutual exclusion and visibility when its structured syntax is sufficient. Choose `ReentrantLock` when timed or interruptible acquisition, multiple Conditions, or an explicitly measured fairness policy matters. Consider a read-write or stamped lock only for a demonstrated read-heavy access pattern, because their protocols and failure modes are more complex.

Use atomics for a narrow state transition with a clear linearization point, preferably publishing an immutable aggregate when several fields belong to one invariant. Expect CAS retries and keep side effects outside the retry loop. Use `LongAdder` for high-contention metrics whose reads need not define a transactional instant. Choose latches, barriers, phasers, semaphores, and blocking queues according to their lifecycle and capacity semantics. Finally, explain timeout, interruption, cleanup, observability, and overload behavior; those details determine whether correct code remains operable in production.

Production Perspective

Common failures include missing unlock or release in finally, fairness enabled without evidence, `StampedLock` optimistic data used before validation, `LongAdder` used for strict limits, a latch never counted down after an exception, and interrupt status swallowed. Contention can appear as high CPU from CAS retry or as blocked time from locks and semaphores.

Use thread dumps, JFR lock events, queue and permit wait metrics, timeout rates, and CPU profiles. Document what each primitive protects and the permitted acquisition order. The primitive is an implementation detail of the invariant, not the invariant itself.

Interview-Ready Summary

`ReentrantLock` adds timed and interruptible acquisition, fairness options, and `Conditions` beyond `synchronized`. Read-write and stamped locks are workload-specific optimizations. Atomic variables use compare-and-set for operations on a narrow state boundary; they do not make multi-object workflows atomic.

`LongAdder` improves highly contended statistics but does not provide an exact transactional sum. Semaphore limits concurrent capacity, `BlockingQueue` coordinates producers and consumers, and latches or barriers coordinate milestones. Every wait needs failure, timeout, interruption, and cleanup semantics.

Revision Notes

- Prefer the simplest primitive that clearly protects the invariant.
- Always unlock ReentrantLock in finally.
- tryLock and lockInterruptibly support bounded or cancellable waiting.
- Fair locks can reduce throughput and are not a default.
- Condition awaits must test predicates in a loop.
- ReadWriteLock helps only for suitable measured read-heavy workloads.
- Do not upgrade a read lock to a write lock casually.
- StampedLock optimistic reads must be validated before use.
- StampedLock is not reentrant.
- Compare-and-set may retry under contention.
- Lock-free does not mean every thread makes bounded progress.
- AtomicReference protects replacement, not mutation of its value.
- Separate atomics do not automatically protect multi-field invariants.
- ABA matters when value history carries meaning.
- LongAdder is designed for contended statistics, not strict balances.
- Semaphore limits simultaneous access, not requests per second.
- Release every permit in finally.
- CountdownLatch is one-shot; barriers coordinate repeated phases.
- BlockingQueue provides safe handoff and can encode bounded capacity.
- Interruption is cooperative and must be propagated or preserved deliberately.
- AQS-based synchronizers combine state with a queue of waiting threads.
- Queue growth, not only hold time, explains lock-related tail latency.
- Fairness is a measured policy and can reduce throughput.
- A signal means a condition may be true; always recheck the predicate.
- Release a read lock before attempting a general read-to-write upgrade.
- Never publish StampedLock optimistic data before successful validation.
- The successful CAS is the linearization point; retries must be side-effect free.
- Lock-free guarantees system progress, not bounded progress for every thread.
- Version state when an A-to-B-to-A history changes correctness.
- Semaphore permits require exact acquire/release accounting.
- Select coordination utilities by lifecycle, participant model, and failure behavior.